HacdsDeFi 白皮书

2026-11-11　第三版（对应合约 #3000）

> 本文档描述 Hacash 主网合约 `HacdsDeFi3000.fitsh`（合约地址 `T1A8kaeWqK6wMUw5UMHCA6tMQ1iBdECyH`，资产 serial 3000）。
> 文中的每一个数字都是合约源码里的常量，部署后不可修改。

写在前面

hacds 是 Hacash 主网上的一个链上资产，serial 3000，decimal 0。

它跑在 Hacash 的第一份 DeFi 合约上。Hacash 主网的 HVM（图灵完备智能合约）在区块高度 765,432 激活，

这份合约是主网上第一个把 DeFi 四件事——兑换做市、借贷、清算、金库——整套做进单份合约的实现。

这一版和上一版的区别不是参数微调，是**把三个中心化部件整个删掉了**：

上一版有	这一版
链下预言机算分 + 签证书，合约验签	合约链上直读钻石基因，自己算分
管理员六个参数函数	没有管理员，一个都没有
`set_paused` 全局暂停开关	没有暂停开关
serial 2000，decimal 8	serial 3000，decimal 0
单笔存 1 颗钻石	单笔存 200 颗钻石

删掉的东西不会再回来。这是设计选择，不是路线图。

本文分六部分：上一版的问题，这一版的设计，四个特点，系统组成，当前状态，已知限制。

一、上一版

1.1 原来的方案

上一版把 hacds 定义成刻在 HACD 钻石上的铭文。方案分四块。

发币靠铭文。每颗 HACD 刻一条 Hacds 铭文，一颗钻石只有一条有效。普通钻石刻一张给 10000 个 hacds，特殊钻石给多少（爱心 50 万、四连字母 30 万、异形 20 万）由社区投票决定。

交易靠插件。去中心化交易所做成一个叫 MoneyNex 的插件钱包，用户装插件、用插件的钱包进出场。

上所当目标。白皮书里写的第一个上所目标是 xeggex.com。

检索靠扫链。每个客户端自己扫全网铭文，第三方服务器可以另外提供检索。

1.2 我们做完了，也上所了

这些不是纸面方案。铭文系统做出来了，MoneyNex 插件跑起来了，hacds 先后上了 xeggex.com 和 nonkyc.io 两家交易所。

然后停牌了。

1.3 问题出在哪

症结在第一条，其余三条都是它的后果。

价格没有锚。发行比例靠投票，等于每类钻石值多少个 hacds 随时可以被投票改掉。交易所拿不到可复现的换算依据，做市商给不出稳定报价，深度也就起不来。

交易环节押在别人身上。去中心化交易所是第三方插件，插件停更就是交易所停摆。中心化交易所更直接，成交和流通都在 xeggex 和 nonkyc 手里。

没有退路。这两家所是全部的流通渠道，它们一停，链上再没有别的地方能交易。

连起来看：币不能自由流通，就没法定价；没法定价，就没法做市；没法做市，就只能依赖交易所；交易所一停就断。

上不上所不是重点。上了所，设计上的毛病也不会消失，只是推迟到停牌那天才暴露。这是这一版改设计的直接原因。

1.4 这一版改了什么

只做了一件事：把币从钻石上拿下来。

hacds 改成标准链上资产，不依附任何钻石，能自由拆分和转账。金融逻辑从插件和交易所搬进合约。流通不靠上所，链上 AMM 自己就能交易，上所以后只是可选项，不再是活下去的前提。

然后在这个基础上，把上一版剩下的两个中心化部件——**预言机**和**管理员**——也删了。

二、这一版的设计

2.1 发行量怎么来的

hacds 的总量不是拍的，是从全网钻石算出来的。

每颗 HACD 按 HIP-5 五维评分（形状、样式、颜色、命名、编号）。合约里 `SCORE_SCALE = 1e8`：**1 分铸 1 亿个 hacds**。总额取全网钻石的精确均值：

```
全网总分数 = 均值评分 × HACD 总发行量
           = 3.55440306 × 16,777,216
           = 59,632,987 分

总供给     = 总分数 × 每分铸 1e8 个
           = 59,632,987 × 100,000,000
           = 5,963,298,700,000,000 个
```

这个数谁都能自己复算：拉一遍浏览器接口，逐颗算分，结果一样。上一版缺的就是这个：一个不依赖任何人拍板的基准。

2.2 评分怎么算：链上直读，没有预言机

上一版的做法是：链下算分 → 签名成证书 → 合约验签 → 链上重算比对。**这一版整套删掉了。**

现在合约在收到 HACD 的同一笔交易里，直接向 VM 要钻石的基因数据：

```
hacd_number_batch(names)        → 每颗钻石的编号（u64）
hacd_visual_gene_batch(names)   → 每颗钻石的视觉基因（10 字节）
```

拿到基因后，合约自己按 HIP-5 的五个维度打分。**全程没有链下参与，没有证书，没有签名，没有可伪造的环节。**

这个能力依赖 Hacash 节点 v1.3.0 起提供的「HACD 基因视图」，并在区块高度 800,000 激活。两个条件缺一不可——节点版本不够拿不到基因，高度不够 VM 不认这个调用。

2.3 批量：一次存 200 颗

这是 v1.3.0 带来的第二个变化。

上一版单笔只能存 1 颗。原因是 gas：FitSH 算一颗钻石的分要烧掉约 3,270 单位的 resource 桶，而单桶上限是 6,100。存 2 颗就超了。

这一版把算分逻辑从 FitSH 挪进了一个 **RV64IM 计算内核**（`score_sum`）：

一次调用传入最多 200 颗钻石的名字、编号、基因，直接吐回总分。

```
输入  names[6n] ‖ numbers[8n] ‖ visual_genes[10n]
输出  Σ score（8 字节大端 u64）
```

算 200 颗只要 3,587 单位 compute，一个桶就装下了。**单笔 200 颗，一次签名。**

存入批量、赎回仍然一次一颗。原因见 §7.1。

2.4 资产规格

项	值
serial	3000
ticket / name	HACDS / hacds
decimal	**0**（1 个最小单位 = 1 个 hacds）
总供给	5,963,298,700,000,000
issuer	T1A8kaeWqK6wMUw5UMHCA6tMQ1iBdECyH（合约自身）
发行成本	8 HAC（协议费，随出块奖励浮动）

链上 #1028 的 ticket 也是 HACDS（它的 name 是 "HACD Fragments"，别人的资产）。协议不检查 name 和 ticket 是否重名，只看 serial 有没有被占。**认 serial 3000。**

> ⚠ 旧版 serial 2000（地址 `VKNwXFFMCEn9XfMrhyHhd5rHqBnEyM3m1`）已停用。
> 两个 serial 是两套独立资产，不可混用、不可换算。

三、四个特点

3.1 开源

合约（fitsh 源码 + RV64IM 内核）、索引器、交易构建服务、钱包（PC + Android），全部源码公开。

DeFi 协议里跑的是别人的钱，用户能自保的唯一办法是自己读得懂代码、算得清账。上一版的钱包是第三方插件，用户只能选择信或不信；这一版把逻辑摆在明面上。

代码公开之后，有三件事可以直接做：

- **评分可独立复算。** 谁都能拉一遍全网钻石、按同一套算法重算评分，看发行量和合约给的对不对。测试网用 119 组差分用例验证过内核与 FitSH 参考实现完全一致。
- **合约行为可审计。** 没有管理员意味着函数清单里不存在任何能改参数、改余额、暂停协议的入口——这一点可以逐个函数验证，不必听我们怎么说。
- **服务可替换。** 索引器和交易构建服务都是无状态 HTTP 服务，任何人可以自己搭一个，把客户端的服务地址改过去即可。

3.2 去中心化

这一条要讲边界。笼统说"完全去中心化"没有意义。

已经做到的：

规则和执行都在链上。AMM 的定价、借贷的放行条件、清算的门槛、手续费的分账，全是合约代码。参与者按同一套规则交易，我们不在中间，也插不进手。

没有管理员。 上一版有六个管理员函数（`set_fee` / `set_loan_rate` / `set_liquidate_bonus` / `set_paused` / `set_oracle` / `rent_batch`）。这一版**一个都没有**，在链上调用会返回 `CallNotExist(51)`——不是"不给你权限"，是**函数根本不存在**。

所有参数都是编译期常量。费率、利率曲线、清算线、储备目标，全部写死在源码里，部署后没有人能改，包括我们。

合约不可升级，没有后门。整个合约没有自毁函数，没有升级函数，没有针对单个用户的冻结函数。部署上去是什么，以后就一直是。

参与无需许可。兑换、做市、借贷、清算、甚至键续租（`rent_keys`），任何人都能来，没有白名单，没有审批。

资产自己拿得住。hacds 是标准链上资产，放在用户自己的地址里。我们不托管，也冻结不了。

还没做到的，一并说清楚：

金库私钥是单点。手续费累积进金库地址 `1FbwjELGMmaomNQq3ZZ6hdgQ17GRGwzAzK`，那是一个冷钱包，只有我们持有私钥。合约里没有提款函数，谁也拿不走用户的资产；但**金库里的协议收入**由这个私钥控制。这是这一版唯一需要信任项目方的地方。

服务端是单点。索引器（借款人台账）和交易构建服务跑在我们的服务器上。它们**不持有任何私钥、不参与共识**，用户可以用自己搭的服务替换，也可以离线签好交易再广播。但它们下线时，网页版的便利功能会受影响。

这两条就是当前去中心化程度的实际边界。

3.3 安全

分两头：用户的钱怎么保管，协议的规则怎么保证。

私钥不触网。 钱包分冷热两端，离线机构建交易并签名，联网机只做查询和广播。私钥只在主进程内存里出现过一次，构建时通过环境变量注入子进程，不落盘。这套流程能成立是因为 Hacash 的交易没有 nonce、没有时间戳过期，离线签好的交易放很久再广播依然有效。

签名在本地做。 交易由随包的 Rust 程序在本地签，不经过任何节点。节点只用来翻译、校验、广播。节点的 `/util/transaction/sign` 接口要求把私钥通过 HTTP 发过去，我们没用它。

钱包文件是加密的。 口令用 scrypt（$N=2^{15}$, r=8, p=1）派生，加密用 AES-256-GCM，AAD 绑定格式版本。PC 与 Android 共用同一份 keystore 格式（`hacds-keystore v1`），可以互相导入。

界面进程没有系统权限。 渲染进程开了 `contextIsolation`、关了 `nodeIntegration`，只能通过 preload 的白名单调主进程。渲染层即使拿到恶意内容，也碰不到文件系统和私钥。

评分不可伪造。 评分在合约链上算，输入是 VM 返回的钻石基因。没有链下环节可以被攻破，也没有签名可以被伪造。

资产守恒。 链上恒等式：用户持有的量加合约持有的量等于总供应。所有操作都保持这个式子，谁都能随时核对。

3.4 DeFi 完备

上一版的"金融"只有两件事：发币，和把币挂到交易所。没有金融原语，不能做市，不能抵押，不能借，收益只能靠涨。

这一版补齐了，一个合约里有四块：

模块	内容
AMM	HAC 和 hacds 双向兑换。恒定乘积 `x × y = k`，手续费 0.5%，**LP 80% / 金库 20%**
借贷	用 hacds 做抵押借 HAC。借款线 200% 抵押率、清算线 160%、利率 2%–46% 浮动
清算	两级：普通清算（105%–160%，单次砍一半债务）+ 深水接管（<105%，全额）
金库	手续费沉淀，任何人可触发清扫进冷钱包

另有几样工具。

原子交易。 多个操作并进同一笔交易，要么全成要么全败，协议上限 200 个动作。

社会化坏账（hcut）。 极端情况下坏账会按比例摊到所有存款人头上，用一个全局系数 `hcut` 表示，只降不升。这是公开的、一次性的、不可逆的损失——不是 bug，是最后一道防线。

存储租期。 Hacash 的合约状态要付租金，键到期会失效。合约提供无许可的 `rent_keys`，任何人都能续租并拿到 0.07 HAC/键的报酬。keeper 就是这么工作的，任何人都可以跑。

利息用惰性计算。 不写定时任务，不依赖 keeper，有人跟合约交互时按块高差把利息补上。省 gas，也不会因为没人维护就停摆。

四、系统组成

整体分四层：链上的合约和资产，服务端的节点与辅助服务，客户端的钱包，以及冷热分离的使用方式。

4.1 三层与职责

层	组件	能做	不能做
① 链上	HacdsDeFi 合约 · `T1A8kaeWqK6wMUw5UMHCA6tMQ1iBdECyH`	按代码规则转移资产、记债务、清算、算分	被任何地址改参数或暂停
① 链上	hacds 资产 · serial 3000 · decimal 0	自由拆分、转账、上交易所	被增发（没有增发函数）
② 服务端	全节点 RPC · `node.hacds.com`	读链、校验、广播	拿走私钥（我们没用它的签名接口）
② 服务端	API 代理 · `api.hacds.com`	转发到节点，补 CORS + HTTPS	改变任何语义
② 服务端	索引器 · `indexer.hacds.com`	扫链记「谁有债务、谁可清算」	影响任何链上状态
② 服务端	构建服务 · `build.hacds.com`	把人类输入转成未签名交易	签名（私钥永远在客户端）
③ 客户端	PC 控制台 / Android	本地签名、广播	绕过合约规则

数据流向：客户端读状态走「API 代理 → 全节点」，读台账走索引器；

要发交易时，可以自己构造，也可以请构建服务生成未签名交易，

签名始终在客户端本地完成。

⚠ **一个硬约束**：全节点的沙箱调用每 IP 并发上限是 2。超出的请求直接拒绝，

不排队。客户端必须串行化 + 退避重试。

为什么服务端地址是子域名而不是 `IP:端口`：Cloudflare Workers 只能访问

80/443 端口，任何 `IP:8082` 形式的地址会在建立连接前被拒绝。

所以对外统一走 `*.hacds.com` 子域名 + Tunnel。

4.1 职责边界

组件	能做	不能做
合约	按代码规则转移资产、记债务、清算	被任何地址修改参数或暂停
全节点	读链、校验、广播	拿走私钥（我们没用它的签名接口）
索引器	记录谁有债务、算健康度	影响任何链上状态
构建服务	把参数转成未签名交易	签名（私钥永远在客户端）

组件	能做	不能做
客户端	本地签名、广播	绕过合约规则

五、当前状态

5.1 已经完成的

项	状态
合约源码	定稿，16,198 字节（上限 16,384），通过完整安全审计
评分内核	RV64IM 内核 2,026 字节，119 组差分测试与 FitSH 参考实现一致
测试网	本地全节点 + 多地址，A/B/C/G 四组 40 项功能测试全通
攻击测试	43 项攻击场景（重入、抢跑、价格操纵、下溢等）
服务端	全节点 / 索引器 / 构建服务 / keeper 全部在线，开机自启
PC 钱包	v0.3.1，Windows 安装包 + 免安装版
Android	v0.2.0，签名 APK，arm64
官网	hacds.com（含白皮书、文档、网页版控制台）

5.2 还没做的：部署

合约尚未部署到主网。 原因只有一个：VM 升级的激活高度还没到。

条件	状态
节点 ≥ v1.3.0（提供 HACD 基因视图）	✅ 已就绪
区块高度 ≥ 800,000	⏳ 当前 798,812，约剩 1,188 块

两个条件缺一不可。高度到了就会部署。

在那之前，官网和钱包都是按 #3000 规格构建的，指向的合约地址在部署后才会有代码。

5.3 测试网验证过什么

- **批量存入**：单笔 54 颗实测成功；差分测试覆盖到 200 颗上限
- **AMM**：恒定乘积精确到个位，滑点保护、首注死份额都正常
- **借贷**：借款额度、利率累积、还款、健康度视图，全部与预期一致
- **清算**：普通清算和深水清算都真实触发过，账目精确闭合
- **转账**：HAC / hacds / HACD 三种资产互转
- **攻击模拟**：43 项全部按预期失败，没有可利用路径

累计发现并修复 2 个真实缺陷（都在清算路径上，测试网的链上真实执行才暴露出来）：

一个 resource 桶超限，一个下溢。修复说明在仓库 `hacds_exchange/修复说明-清算gas与下溢.md`。

六、已知限制

有几件事不是没做，是权衡之后先这样。写在这里，免得后面的人踩。

6.1 赎回仍然一次一颗

存入能批量 200 颗，赎回只能 1 颗。这是有意的。

赎回要给「挑钻石溢价」定价：赎回高分钻石得多付钱，费率取决于这颗钻石的分数相对**池子平均分**的位置。批量赎回时每一颗的判断都不同，而且赎回会触发 AMM 自动补足，每颗都要单独判余额。

更关键的是**账目口径**：批量赎回如果在中途某颗失败，前面已处理的部分怎么回滚？合约不可升级，这种回滚逻辑写错就是资金死锁。

所以：**存入批量，赎回单颗。** 钱包会自动把你的选择拆成多笔，用户点一次即可。

6.2 金库是单点

冷钱包，不换。这是项目方的收入账户，也是唯一需要信任我们一下的地方。

6.3 服务端是单点

索引器和构建服务由我们运行。它们不能动链上状态，但下线时网页版会不方便。任何人都能自己搭一份——源码是公开的。

6.4 出块 300 秒

广播成功不等于上链，`ret:0` 只是进了内存池。钱包按出块节奏补刷几次（15、45、90、180、320 秒）。

6.5 利息不单独显示

合约惰性计息，计息的起点没有对外接口，算不出当前的应付利息，只在还款时显示链上结算的结果。

6.6 取款可能暂时被拒

如果借出去的钱太多、现金不够，存 HAC 的人取款会被拒绝。这是故意的——协议宁可让你等，也不写一个会死锁的排队系统。好在两层保护让这种情况很少发生：借款总量上限 50%，AMM 的 HAC 只有 30% 可出借。

6.7 合约不可升级

发现 bug 也改不了。这就是为什么参数设计得保守（20% 清算缓冲、50% 借款上限、两层现金保护），也是一个理由：**别把全部身家放进来。**

七、常见问题

Q：hacds 会增发吗？

不会。按钻石等值发行，总量固定为 5,963,298,700,000,000 个，合约里没有增发函数。存入时合约会检查：`已发行 + 本次铸造 ≤ 总供给`，超了直接拒绝（return 53）。

Q：有没有管理员能拿走我的钱?

没有管理员。合约里不存在任何 `set_*` 函数，调用会返回 `CallNotExist(51)`——是函数真的不存在，不是权限不足。任何人都无法暂停、修改参数、或者转走用户资产。

Q：一次能存几颗钻石?

200 颗。一次交易、一次签名。

Q：为什么赎回只能一颗?

见 §6.1。挑钻石溢价要逐颗定价，批量回滚逻辑在不可升级的合约里风险太高。

Q：APY 是多少?

浮动的。存款收益取决于借出去多少，借款成本在 2%–46% 之间浮动。界面会显示实时值。

Q：钻石跌了怎么办?

跌到清算线以下，清算人会替你还债并拿走部分抵押。剩下的还是你的。

没有宽限期——跌破 160% 就随时可被清算，不会提前通知。单次清算只砍一半债务，但价格继续跌会被反复清算；跌破 105% 会被一次清完。

跌之前补抵押或者还一部分，就不会被清算。

Q：清算会不会一次清光我?

普通清算（105%–160%）一次最多砍 50% 债务，而且清算奖励是**按还款额**算的，不是按债务比例。所以清算掉一半债务只会拿走约三分之一的抵押——清完抵押率反而升到 200% 以上，立刻脱离危险区。

但价格继续跌的话会被再次清算。跌破 105% 进入深水区，就是一次清完全部债务。

Q：为什么赎回高分钻石要多付钱?

防止有人把池子里的好钻石挑走。这是对「挑拣行为」的税，最高 30%，普通和低分钻石不收。

Q：协议赚的钱去哪了?

金库冷钱包 `1FbwjELGMmaomNQq3ZZ6hdgQ17GRGwzAzK`。地址公开，任何人都能查余额。

Q：会不会跑路?

合约里没有提款函数，没有管理员权限，代码公开可审计。但**金库私钥是单点**——这是唯一需要信任项目方的地方。另外合约不可升级，所以也别把全部身家放进来。

Q：钻石价格数据从哪来?

不用外部数据源。直接读 AMM 池子的余额比例，没有人能伪造。单笔交易最多让参考价移动 10%，砸盘操纵非常昂贵。

Q：键到期了会怎样?

Hacash 的合约状态需要付租金，键到期会失效（`StorageNotActive`）。合约提供无许可的 `rent_keys`：任何人都能续租，还能拿到 0.07 HAC/键的报酬。keeper 就是这么工作的。

附录 A：地址与参数

项	值
部署者	`15fv6cNEapTNqYhigMPr38pC1XRKn3Wh5q`
合约 nonce	2
合约地址	**`T1A8kaeWqK6wMUw5UMHCA6tMQ1iBdECyH`**
金库（冷钱包）	`1FbwjELGMmaomNQq3ZZ6hdgQ17GRGwzAzK`
hacds serial	**3000**（decimal **0**）
hacds 总供给	5,963,298,700,000,000
`SCORE_SCALE`	1e8（1 分 = 1 亿个 hacds）
全网总分数	59,632,987 分（= 均值 3.55440306 × 16,777,216 颗）
`FEE_BPS`	50（AMM 手续费 0.5%）
手续费分配	LP 80% / 金库 20%
`BORROW_RATIO_BPS`	20,000（借款线：200% 抵押率，即最多借 50%）
`LIQ_COLL_RATIO_BPS`	16,000（清算线：160%）
`CLOSE_FACTOR_BPS`	5,000（普通清算单次上限 50% 债务）
`DEEP_TRIGGER_BPS`	10,500（跌破 105% → 深水全额清算）
`LIQ_BONUS_BPS`	500（清算奖励 5%，按还款额计）
`LIQ_PROTOCOL_BPS`	100（其中 1% 进储备）
利率曲线	U=0 → 2%，U=35% → 6%，U=50% → 46%
`LENDABLE_AMM_BPS`	3,000（AMM 的 HAC 仅 30% 可出借）
`U_MAX_BPS`	5,000（全局利用率上限 50%）
`BORROWER_CAP_BPS`	2,000（单借款人上限 20%）
`RES_TARGET_BPS`	300（坏账储备上限 3%）
`MAX_DEPOSIT_COUNT`	200（单笔存入上限）
`RENT_PERIOD`	300 块
`REF_WINDOW`	300 块（EMA 时间常数，约 1 天）
`EMA_MAX_STEP_BPS`	1,000（参考价单次最大移动 10%）
`REF_DEV_BPS`	2,000（熔断阈值：现货偏离参考价 20% → 暂停新增借款）
VM 三桶上限	compute 18,009 / resource 6,100 / storage 111,911
交易上限	16,384 字节

项	值
出块目标	300 秒

附录 B：单位

hacds 是 **decimal 0** 的资产。链上那个整数就是个数，1 就是 1 个，不存在小数。这比上一版（decimal 8）简单——不用在 atoms 和「个」之间来回换算。

三条规矩：

1. **hacds 一律用个数，不换算法。** 余额、池子、抵押、输入框、报价，看到的都是同一个数字。
2. **HAC 是另一套。** 链上单位是 zhu（1 HAC = 1e8 zhu），界面上一律用 HAC。所以 `add_liquidity(hac_amt, ds_amt)` 两个参数的量纲不一样，一个是 zhu，一个是 hacds 个数。这是最容易犯 1e8 倍错误的地方，我们自己在首注时就犯过一次。
3. **总供给 5,963,298,700,000,000 超过 JS `Number.MAX_SAFE_INTEGER` 的 66%**，JS 里用 BigInt，Kotlin 里用 BigInteger，`Long` 不够。

本文档对应合约 `HacdsDeFi3000.fitsh` 定稿参数。